BI-DIRECTIONAL SIGN LANGUAGE TO AUDIO (TAMIL (TAMIL & ENGLISH) AND TEXT TO SIGN LANGUAGE ASSISTIVE SUITE

AD23505 SOCIETAL ORIENTED PROJECT

A PROJECT REPORT

Submitted by

Hepsiba Selvi M	**2024510021**
Vikashni V	**2024510002**

Under the supervision of
C. Sunil Retmin Raj

DEPARTMENT OF INFORMATION TECHNOLOGY

MADRAS INSTITUTE OF TECHNOLOGY CAMPUS

ANNA UNIVERSITY, CHENNAI – 600044

JULY 2026 - OCTOBER 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report titled **"Bi-Directional Sign Language to Audio (Tamil & English) and Text to Sign Language Assistive Suite"** is the bonafide work of **Hepsiba Selvi M (2024510021)** and **Vikashni V (2024510002),** who carried out the project work under my supervision.

Signature

Dr. Radha Senthilkumar

HEAD OF THE DEPARTMENT

Professor

Department of Information Technology

MIT Campus, Anna University

Chennai – 600044

Signature

C. Sunil Retmin Raj

SUPERVISOR

Professor

Department of Information Technology

MIT Campus, Anna University

Chennai – 600044

ACKNOWLEDGEMENT

It is essential to mention the names of the people, whose guidance and encouragement made us accomplish this project.

We express our thankfulness to our project supervisor **C. Sunil Retmin Raj**, Head of the Department, Department of Information Technology, MIT Campus, for providing invaluable support and assistance with encouragement which aided to complete this project.

We express our gratitude and sincere thanks to our respected Dean of MIT Campus, **P. Jayashree**, for providing excellent computing facilities throughout the project.

Vikashni V 2024510002

Hepsiba Selvi M 2024510021

ABSTRACT

This project presents a bi-directional assistive communication suite designed to reduce communication barriers between Deaf or hard-of-hearing people and hearing people. Unlike systems that focus only on recognising signs or only on presenting text, the proposed suite combines two complementary workflows: sign-to-audio and text/voice-to-sign. In the first workflow, a standard RGB webcam captures hand movements. MediaPipe Hands is used to locate 21 hand landmarks, and the resulting coordinates are normalised to reduce the effect of hand position and scale. A hybrid recognition approach, described in the repository as a Random Forest model combined with geometric rules, classifies supported gestures. A hold-to-confirm sentence builder is intended to reduce accidental additions caused by brief or unstable predictions. The recognised sequence is passed to a bilingual layer and spoken through text-to-speech engines in English or Tamil. In the reverse workflow, a user enters Tamil or English text or provides microphone input. Speech is converted to text, the text is tokenised and transliterated where appropriate, and tokens are mapped to available sign representations or fingerspelling. The application presents hand-skeleton diagrams and sequential sign cards with playback controls, allowing the recipient to follow the message one sign at a time. The repository describes several interfaces: a desktop hub, a Streamlit dashboard, an OpenCV webcam interface and a dedicated text-to-sign player. The design uses accessible consumer hardware and common Python computer-vision and machine-learning libraries, avoiding the need for specialised gloves. The project is relevant to everyday exchanges in education, public services, healthcare and home settings, particularly where Tamil and English are used. However, the prototype should be understood as a communication aid rather than a replacement for qualified interpreters. Sign languages have their own grammar, regional variation, facial expressions and movement patterns; a limited gesture vocabulary and word-by-word mapping cannot represent every sentence accurately. The repository's reported model metrics should be validated independently using a documented, representative test set and real users. This report explains the problem, objectives, requirements, architecture, algorithms, implementation modules, setup, testing strategy, limitations, ethical considerations and future improvements of the system.

Keywords: assistive technology, sign language recognition, Tamil, English, MediaPipe, computer vision, Random Forest, speech synthesis, speech-to-text, accessibility.

TABLE OF CONTENTS

Abstract i
List of Figures ii
List of Abbreviations iii
1. Introduction 1
1.1 Background and Context 1
1.2 Problem Statement 2
1.3 Motivation and Social Relevance 2
1.4 Aim and Objectives 3
1.5 Scope of the Project 3
1.6 Expected Contributions 4
2. Literature and Technology Review 5
2.1 Assistive Communication Systems 5
2.2 Sign Language Recognition 5
2.3 Hand Landmark Detection 6
2.4 Classification and Gesture Stability 6
2.5 Speech Recognition and Synthesis 7
2.6 Bilingual and Regional-Language Support 7
2.7 Gap Addressed by the Project 8
3. System Requirements and Feasibility 9
3.1 Functional Requirements 9
3.2 Non-Functional Requirements 10
3.3 Hardware and Software Requirements 10
3.4 Feasibility Analysis 11
4. System Architecture and Design 12
4.1 Overall Architecture 12
4.2 Sign-to-Audio Pipeline 13
4.3 Text/Voice-to-Sign Pipeline 14
4.4 Data Flow and Module Design 15
4.5 User Interface Design 16
5. Methodology and Algorithms 17
5.1 Landmark Extraction and Normalisation 17
5.2 Gesture Classification 18
5.3 Sentence Builder and Debouncing 19
5.4 Bilingual Processing 19
5.5 Speech-to-Text and Text-to-Speech 20
5.6 Sign Rendering and Playback 21
6. Implementation Details 22
6.1 Repository Structure 22
6.2 Core Modules 22
6.3 Application Entry Points 23
6.4 Installation and Execution 24
7. Testing and Evaluation 25

7.1 Test Strategy 25
7.2 Test Cases 26
7.3 Evaluation Metrics 27
7.4 Results Interpretation 27
8. Applications, Benefits and Impact 28
9. Limitations, Risks and Ethics 29
10. Future Enhancements 30
11. Conclusion 31
References 32
Appendix A: Sample User Workflow 33
Appendix B: Suggested Test Checklist 34

LIST OF FIGURES

Figure 4.1: High-level architecture of the assistive suite

Figure 4.2: Sign-to-audio processing pipeline

Figure 4.3: Text/voice-to-sign processing pipeline

Figure 4.4: Main software modules and data flow

Figure 5.1: Conceptual hand-landmark representation

Figure 5.2: Gesture stabilisation and sentence-building logic

Figure 6.1: Repository directory structure

Figure 6.2: Conceptual text-to-sign player layout

Figure 7.1: End-to-end testing workflow

Note: The figures listed above are conceptual figures recommended for inclusion. The report describes the architecture from the repository documentation; screenshots should be captured from a running application before final academic submission.

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
ASL	American Sign Language
BSL	British Sign Language
CV	Computer Vision
GUI	Graphical User Interface
HUD	Heads-Up Display
ML	Machine Learning
RGB	Red, Green, Blue
STT	Speech-to-Text
TTS	Text-to-Speech
UI	User Interface
WFD	World Federation of the Deaf
API	Application Programming Interface
FPS	Frames Per Second
CPU	Central Processing Unit
GPU	Graphics Processing Unit
IDE	Integrated Development Environment
NLP	Natural Language Processing
OS	Operating System
CSV	Comma-Separated Values

1. INTRODUCTION

1.1 Background and Context

Communication is a basic requirement for participation in education, employment, healthcare, public services and social life. Deaf and hard-of-hearing people may communicate through a sign language, written language, speech, lip-reading or a combination of methods. Hearing people who do not know sign language may find it difficult to communicate directly with a signing person. The result can be dependence on an interpreter, a family member or written exchanges, none of which is available in every situation.

Assistive communication technology attempts to reduce this gap by converting information from one form into another. Examples include speech recognition, text-to-speech, captioning, sign recognition and sign-display systems. Each technique solves only part of the problem. A sign-recognition application may recognise a gesture and produce text, but it does not necessarily help the hearing person answer in a form that the Deaf person can understand. Conversely, a text-to-sign display may help with incoming messages but may not allow a signing person to respond using their own gestures.

The repository describes a bi-directional suite that brings these two directions into one project. It supports a webcam-based sign-to-audio workflow and a text/voice-to-sign workflow, with English and Tamil as the principal spoken or written languages. This report treats the repository README and its accompanying project-report document as the source of the stated design. Details such as model performance are therefore described as repository-reported claims unless independently tested.

1.2 Problem Statement

The problem addressed is the communication gap that occurs when a Deaf or hard-of-hearing person and a hearing person do not share a communication method. The gap becomes more pronounced when a system supports only English or only one direction of communication. In a real conversation, both participants need to understand incoming information and express responses.

The project aims to provide an affordable prototype that can: (i) observe a supported hand gesture through a regular camera and map it to a label; (ii) combine recognised labels into a sentence; (iii) produce audible English or Tamil output; and (iv) transform typed or recognised Tamil/English text into a sequence of visual sign representations. The system is not a full translator for every sign language. It is a bounded assistive tool whose reliability depends on vocabulary coverage, the quality of the gesture dataset, speech recognition and the accuracy of the mapping between words and signs.

1.3 Motivation and Social Relevance

The motivation is practical inclusion. In a classroom, a student may need to understand a short instruction. At a service counter, a customer may need to communicate a simple request. At home, a family member may want to exchange everyday messages. A low-cost tool that makes a limited set of common expressions available in both directions can help initiate communication and encourage awareness of accessibility.

Tamil support is important because language access is not achieved simply by adding English text to an application. Users may be more comfortable reading or hearing a regional language. The project therefore includes Tamil script, transliteration and speech output as part of its design. This does not mean that Tamil speech and Tamil sign language are equivalent: sign languages have their own grammar and are not merely signed versions of spoken languages. That distinction must be reflected in the interface and in any claims made about translation quality.

1.4 Aim and Objectives

The main aim is to develop a prototype assistive communication platform that supports two-way interaction between signing users and hearing users through visual gesture recognition, bilingual speech output and visual sign sequences.

The specific objectives are:

1.5 Scope of the Project

The scope includes a prototype for common supported gestures and short messages. It uses a standard RGB camera for sign recognition and a microphone for speech input. The software stack described by the repository includes Python, MediaPipe, OpenCV, scikit-learn, Streamlit and speech-synthesis components such as pyttsx3 and gTTS. It provides multiple application entry points: a desktop GUI, a Streamlit dashboard, a webcam HUD and a dedicated text-to-sign player.

The scope does not include unrestricted translation of natural sign-language conversations, recognition of every sign, robust recognition under every lighting or camera condition, or certified use in high-stakes settings. The README lists examples such as HELLO, THANK YOU, YES, NO, HELP, WATER, PLEASE, FOOD and HOSPITAL, along with an A–Z fingerspelling set. The precise behaviour of every feature should be confirmed against the installed version and its tests.

1.6 Expected Contributions

The main contribution is integration: several accessible input and output mechanisms are organised within one application family. The forward path links landmark extraction, gesture classification, sentence construction, bilingual processing and speech synthesis. The reverse path links text or speech input, tokenisation, sign mapping, visual display and playback. The project also demonstrates how to build a prototype using consumer hardware rather than dedicated sensor gloves.

A second contribution is its emphasis on regional-language access. Tamil and English are treated as supported interaction languages, and the UI can show Tamil script alongside transliteration. A third contribution is modularity: detection, classification, translation, sentence building, speech synthesis and text-to-sign rendering are represented as separate source modules. This modular design makes debugging and future replacement of individual components easier.

2. LITERATURE AND TECHNOLOGY REVIEW

2.1 Assistive Communication Systems

Assistive communication systems span several categories. Captioning systems convert speech to text; speech synthesis converts text to audio; gesture-recognition systems map observed movements to labels; and sign-generation systems display a visual representation of a message. Combining these capabilities can make an interface more useful, but integration alone does not guarantee correct communication. Every stage can introduce errors, and those errors can accumulate across the pipeline.

A useful design principle is to preserve user control. A person should be able to review a recognised message, correct an error, repeat a sign sequence or choose another language. For an academic prototype, the system should also make clear which features are implemented and which are planned or limited to a demonstration vocabulary.

2.2 Sign Language Recognition

Camera-based sign recognition typically transforms image data into a representation that a classifier can process. A direct image model may learn from pixels, while a landmark-based system first extracts points representing the hand joints and fingertips. Landmark-based methods reduce the dimensionality of the input and can make geometric relationships easier to interpret. They can also fail when the hand is occluded, outside the frame, poorly illuminated or oriented in an unfamiliar direction.

Signs may be static or dynamic. Static handshapes can sometimes be described by one frame or a short stable sequence. Dynamic signs depend on movement, direction, repetition, location relative to the body and timing. A pipeline based mainly on hand landmarks must therefore be careful not to imply that all signs can be identified from a single hand configuration.

2.3 Hand Landmark Detection

The repository uses MediaPipe Hands to estimate 21 landmarks for a detected hand. These landmarks represent key points around the wrist, palm, fingers and fingertips. The coordinates can be represented as a vector and supplied to a machine-learning classifier. A landmark-based approach is attractive for a student project because it can work with a standard RGB camera and avoids the need for instrumented gloves.

Landmark quality is a prerequisite for downstream recognition. If the detector swaps hands, loses a fingertip or estimates a joint inaccurately, the classifier receives distorted features. Testing should therefore include different hand sizes, distances, backgrounds, lighting conditions and camera angles. The system should also have a defined behaviour when no hand is detected: it should not append a guessed sign to the sentence.

2.4 Classification and Gesture Stability

A Random Forest is an ensemble of decision trees that combines their predictions. It can handle a moderate number of numeric features and is often straightforward to train with scikit-learn. The repository also describes geometric heuristics, which can encode rules about finger extension, relative distances or hand configuration. Combining learned classification and explicit rules can be useful when a small vocabulary includes gestures with clear geometric differences.

A major issue in live recognition is prediction flicker. Adjacent video frames may produce slightly different labels even when the user is holding the same gesture. The repository describes a hold-to-confirm sentence builder. In principle, a candidate label must remain stable for a short duration or number of frames before it is accepted. This reduces accidental repeats, but the confirmation interval must balance responsiveness and reliability. The exact threshold and algorithm should be inspected in the source code and measured experimentally.

2.5 Speech Recognition and Synthesis

Speech-to-text converts microphone audio into text. The project documents microphone modes associated with Tamil (ta-IN) and English (en-US). Recognition quality may depend on pronunciation, background noise, microphone quality, speaking speed and the service or library used by the application. When a speech recogniser is uncertain, the interface should allow the user to edit the transcript before generating signs.

Text-to-speech converts text into audible output. The repository names pyttsx3 and gTTS in the speech-output path. pyttsx3 can use locally installed speech engines, while gTTS commonly depends on an online service. The deployment environment therefore affects availability, latency, voice quality and privacy. Tamil voice availability also varies by operating system and engine. A proper evaluation should verify each language separately rather than assuming that a successful English test proves Tamil support.

2.6 Bilingual and Regional-Language Support

Bilingual applications must account for script, pronunciation, token boundaries and language selection. Tamil text may be processed as Unicode characters, while transliteration represents Tamil pronunciation using Latin characters. Transliteration can help display pronunciation but is not the same as translation. Similarly, a Tamil word and an English word may have different grammatical roles and may not map one-to-one to a sign.

The repository describes a bilingual tokenizer and transliteration stage. A practical prototype can map supported words to pre-rendered cards and use fingerspelling for letters or unknown vocabulary. Such a fallback is useful, but the interface should label it honestly: fingerspelling a word is not always the same as producing a natural sign-language equivalent. More advanced translation would require a sign-language lexicon, grammar-aware representation and evaluation by fluent signers.

2.7 Gap Addressed by the Project

The project targets three connected gaps: one-way interaction, limited regional-language support and the lack of a single demonstration interface for both workflows. Its design makes it possible to move from a supported gesture to spoken output and from text or speech to visual sign sequences. The value of the approach should be assessed in terms of task completion, correctness, latency, ease of use and user feedback—not merely whether the application launches or produces an animation.

The repository is best understood as a prototype and learning platform. Its intended social value is credible, but its coverage and accuracy must be established through testing. The report consequently separates the architecture described in the repository from proposed evaluation procedures and does not treat unverified performance statements as independent experimental findings.

3. SYSTEM REQUIREMENTS AND FEASIBILITY

3.1 Functional Requirements

Functional requirements describe what the system must do. The core requirements are: capture live webcam frames; detect a hand and extract landmarks; classify supported gestures; provide a stable mechanism for accepting a gesture; maintain an editable sentence buffer; allow English or Tamil speech output; accept typed text in either language; capture microphone input; convert speech to text; split input into tokens; map supported words or letters to visual sign cards; and provide playback navigation.

The repository also lists a dictionary or flashcard interface, a speech-synthesis testing area and a dashboard describing the architecture and dataset metrics. The four documented launch options provide different ways to access these functions. Feature parity should not be assumed: a feature may be available in one interface and not another, so each entry point needs its own test checklist.

3.2 Non-Functional Requirements

Usability: the interface should show the current input language, recognised content, playback state and available corrections. Responsiveness: live video should update smoothly enough for a user to hold and confirm a gesture. Reliability: an absent or low-confidence hand detection should not silently create a word. Compatibility: the project should document supported Python and dependency versions. Maintainability: modules should have clear responsibilities and useful error messages. Accessibility: controls should be keyboard-operable where possible, text should be legible, and important status should not rely only on colour or sound.

Privacy and security are also requirements. Camera and microphone access should be clearly initiated by the user. If audio is sent to an online speech service, that behaviour should be disclosed. Temporary audio, transcripts and cached speech should not be retained longer than needed. A production deployment would require a privacy review and consent process.

3.3 Hardware and Software Requirements

The README recommends Python 3.11 for compatibility with OpenCV and MediaPipe. A standard webcam is required for sign-to-audio; a microphone is required for voice-to-sign; speakers or headphones are needed for audible output. A general-purpose computer is sufficient for the documented prototype, and the README does not require a dedicated GPU or sensor gloves.

The software dependencies are installed from requirements.txt. The repository describes MediaPipe, OpenCV, scikit-learn, Streamlit, pyttsx3 and gTTS. Actual package versions should be pinned and checked on the target machine. A virtual environment is recommended so that dependencies do not conflict with other Python projects. Online components require network connectivity; offline availability depends on the chosen speech engine and local voice packs.

3.4 Feasibility Analysis

Technical feasibility is reasonable for a limited-vocabulary prototype because hand landmark detection, traditional classification and Python GUI frameworks are readily available. Operational feasibility depends on clear instructions, a suitable camera and a vocabulary that matches the user’s needs. Economic feasibility is favourable for a demonstration because standard consumer devices can be used; however, internet data, maintenance and user testing still have costs.

Schedule feasibility is improved by the modular structure. The system can be developed incrementally: first verify landmark detection, then train and test a classifier, then add sentence construction, speech output and the reverse workflow. The principal risk is scope expansion. Natural sign-language translation is a substantially

larger problem than recognising a small set of static gestures. A realistic project plan should prioritise a transparent and tested subset before adding vocabulary or interfaces.

4. SYSTEM ARCHITECTURE AND DESIGN

4.1 Overall Architecture

The system consists of two main pipelines sharing language resources and presentation components. Pipeline A translates a signing user's supported gestures into speech for a hearing user. Pipeline B converts a hearing user's typed text or microphone speech into a visual sequence for a signing user. Both pipelines use a user interface to display state and provide controls, while shared modules manage language conversion and audio playback.

Figure 4.1. High-level architecture of the assistive suite



Figure 4.1. High-level architecture of the bidirectional assistive suite.

Conceptual Figure 4.1 can be drawn as two horizontal lanes. Lane A is: Webcam → MediaPipe landmarks → normalisation → Random Forest/geometric classifier → hold-to-confirm sentence builder → bilingual text mapping → TTS output. Lane B is: Typed text or microphone → speech-to-text when required → tokenizer/transliteration → sign/fingerspelling lookup → hand diagram renderer → timed sequence player. Error handling and user correction should be shown as feedback loops at the recognition and token-mapping stages.

4.2 Sign-to-Audio Pipeline

The user opens a webcam-based interface and positions the hand within the camera view. The frame is processed to detect a hand and estimate 21 landmarks. Landmark coordinates are transformed into a stable feature vector so that translation or scale changes have less influence on classification. The feature vector is passed to the gesture classifier, which predicts a label from the trained vocabulary and/or geometric rules.

A sentence builder filters unstable predictions. Once a gesture has remained stable long enough, its label is accepted into the current sentence. The user can add spaces, delete the last item, clear the sentence or trigger speech using the available controls. The bilingual layer prepares text for the selected output language. Finally, the TTS engine generates or plays the spoken result. Each stage should provide visible status so that the user can distinguish a missing hand, an unrecognised gesture, a pending confirmation and a completed sentence.

4.3 Text/Voice-to-Sign Pipeline

The user selects English or Tamil and either types a message or activates the appropriate microphone mode. If audio is provided, speech recognition produces a text transcript. The user should be allowed to review and correct the transcript before it is converted. The tokenizer then divides the input into units that can be looked up

in the supported dictionary. The transliteration layer may provide a pronunciation aid or help with Tamil/English token handling.

For each token, the application retrieves a matching sign diagram or fingerspelling representation. The sequence is presented in order, with a highlighted current step and controls such as play, pause, next, previous and speed adjustment. The repository also describes synchronised bilingual voice pronunciation. This audio can assist a hearing user or support demonstration, but the visual signs remain the main output for the Deaf recipient. When a token is unsupported, the application should clearly indicate the fallback rather than presenting an unrelated sign.

Figure 4.3. Text/voice-to-sign workflow

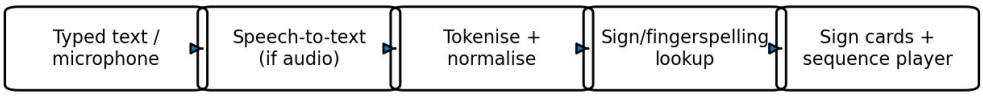


Figure 4.3. Text/voice-to-sign processing pipeline.

4.4 Data Flow and Module Design

The repository separates core logic into source modules. hand_detector.py is responsible for hand tracking and landmark normalisation. gesture_classifier.py contains the model and geometric classification logic. sentence_builder.py manages stable predictions and sentence state. translator.py supports English/Tamil mapping and phonetic handling. tts_engine.py handles speech synthesis. text_to_sign.py provides bilingual tokenisation, speech-to-text integration and sign rendering.

The training workflow is represented by train_model.py, while collect_data.py supports gathering custom gesture examples. The model and dataset are stored separately from source code. This separation is helpful because model files can be retrained without rewriting the user interface. A clear data contract between modules is important: landmark vectors should have a consistent feature order, labels should use one canonical spelling, and text-to-sign lookup keys should be normalised before matching.

Figure 4.4. Main software modules and data flow



Figure 4.4. Main software modules and data flow.

4.5 User Interface Design

The README describes four user-facing applications. The unified desktop hub combines sign-to-audio and text/voice-to-sign in tabs. The Streamlit dashboard provides a browser-based presentation, including the dictionary and speech-synthesis testing features. The OpenCV HUD focuses on real-time webcam interaction and keyboard shortcuts. The dedicated text-to-sign player focuses on sequential sign playback.

The UI should reduce ambiguity. The current language should be visible; the transcript or sentence should remain readable; and controls should have plain labels. The sign player should show one current sign prominently and make upcoming signs visible in the timeline. Keyboard shortcuts should be displayed in the interface rather than requiring users to memorise them. For academic demonstration, screenshots of each running interface should be inserted as Figures 6.2 onwards, with captions that state the exact feature shown.

Figure 6.2. Conceptual text-to-sign player layout

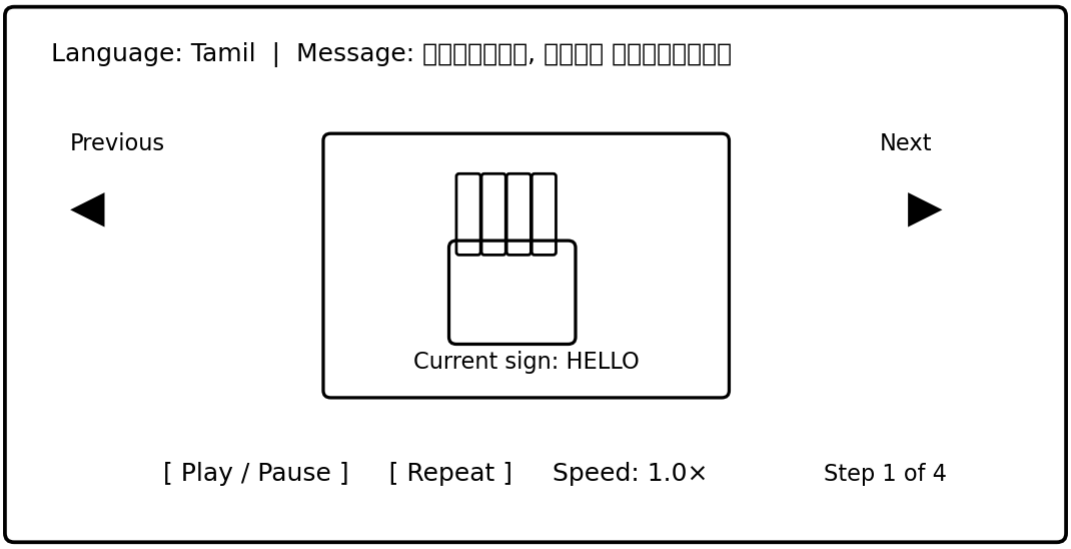


Figure 6.2. Conceptual text-to-sign player layout (mock-up).

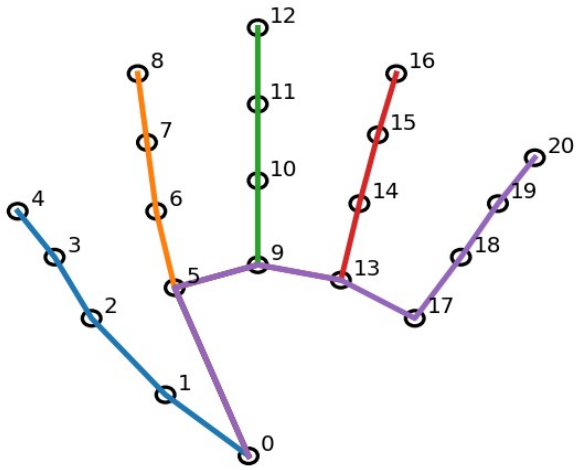
5. METHODOLOGY AND ALGORITHMS

5.1 Landmark Extraction and Normalisation

Let a detected hand contain N = 21 landmarks, each represented by coordinates (x_i, y_i, z_i). The detector outputs a set of points describing the wrist, joints and fingertips. A simple feature vector can be formed by concatenating the coordinates. If all coordinates are used directly, the same handshape may produce different values when the hand moves closer to the camera or to another part of the image.

Normalisation attempts to reduce this nuisance variation. One common approach is to subtract the wrist position from every landmark, producing coordinates relative to the wrist. A scale factor based on a hand span or maximum coordinate distance can then divide the relative values. The README describes invariant normalisation for scale and offset; the precise implementation should be checked in hand_detector.py. Normalisation must be consistent between training and live inference. If training features and runtime features use different transformations, the classifier's predictions will degrade.

Figure 5.1. Conceptual hand-landmark representation



Schematic only: landmark indices shown for explanation; exact coordinates come from the detector.

Figure 5.1. Conceptual hand-landmark representation.

5.2 Gesture Classification

The repository describes a hybrid classifier combining a Random Forest model with geometric rules. During training, labelled landmark examples are used to learn the relationship between hand features and gesture classes. The trained model is stored in a pickle file under the model directory, while feature data is stored under data. At runtime, a normalised feature vector is passed to the model to obtain a class prediction.

A geometric classifier can provide additional decisions based on interpretable relationships, such as whether a finger is extended or whether fingertips are close to one another. The combination can be useful when the vocabulary is small, but the precedence between the machine-learning model and rules must be explicit. The

system should also handle unknown gestures. A forced prediction for every hand configuration can lead to confident-looking but incorrect speech, which is particularly risky for emergency-related vocabulary.

5.3 Sentence Builder and Debouncing

The sentence builder addresses the temporal instability of frame-by-frame classification. Instead of accepting each frame independently, the application tracks the candidate label over a sequence of frames or a hold duration. If the candidate remains stable long enough, it is accepted once. If the label changes too quickly, the candidate timer resets or the prediction is discarded. A cooldown or release condition may be needed to prevent the same held gesture from being appended repeatedly.

Figure 5.2. Gesture stabilisation and sentence-building logic

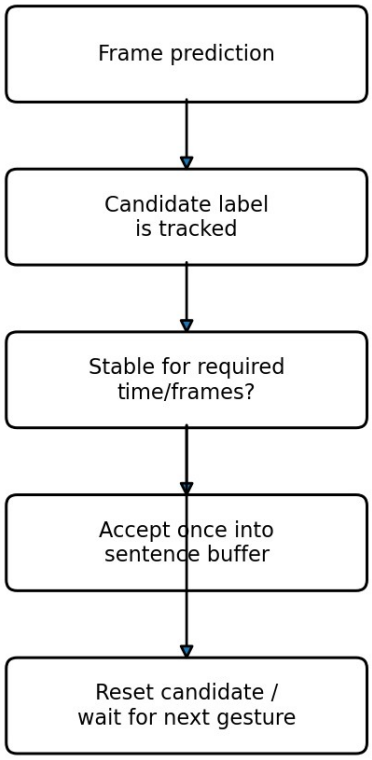


Figure 5.2. Gesture stabilisation and sentence-building logic.

The builder also manages spaces, deletion and clearing. These operations are essential because a recognition error should be recoverable without restarting the application. Testing should measure both false acceptance and missed acceptance: a very short threshold may accept noise, while a very long threshold can make the interface feel slow. The appropriate value should be selected empirically across multiple users and gestures.

5.4 Bilingual Processing

Bilingual processing coordinates English labels, Tamil script and transliterated pronunciation. It may involve normalising case, trimming whitespace, tokenising sentences and mapping known words to their language-specific representation. The README provides examples such as HELLO / வணக்கம் / Vanakkam and HELP / உதவி / Uthavi. These entries illustrate how one concept can be displayed with an English label, Tamil script and a pronunciation aid.

A robust dictionary should store these fields explicitly instead of relying on ad hoc string substitutions. It should distinguish a concept identifier from its English and Tamil surface forms. If a word has no direct equivalent or is not supported, the interface should fall back to spelling or ask the user to rephrase. Translation must be reviewed by fluent Tamil speakers and sign-language users, particularly for signs whose meaning depends on context.

5.5 Speech-to-Text and Text-to-Speech

For speech input, the system records audio from the selected microphone and invokes a recogniser configured for the intended language. The output is text, which then passes through the same tokenisation path as typed input. The user should be able to inspect the transcript because an incorrect recognised name or number can change the meaning of a message. Speech recognition performance should be measured separately for Tamil and English, in quiet and noisy conditions.

For speech output, the selected text is passed to a TTS engine. pyttsx3 may use local speech engines, whereas gTTS generally requires an internet connection. The application should handle missing voices, network errors and delayed synthesis. If speech is generated asynchronously, UI controls should remain responsive and audio requests should not overlap unpredictably. Tests should verify that the generated speech corresponds to the visible sentence and that language selection is respected.

5.6 Sign Rendering and Playback

The reverse pipeline uses a sign sequence and a hand skeleton diagram synthesiser. The repository lists pre-rendered anatomical hand gesture diagram cards in assets/signs and describes procedural hand skeleton configurations. These representations can help demonstrate the sequence of handshapes, but handshape illustrations alone may not capture movement, facial expression, body location or grammatical features required for a natural sign.

The player should maintain an ordered list of tokens and their corresponding visual resources. Playback state includes the current index, play/pause status and speed. Next and previous controls allow manual navigation; the timeline highlights the active step. If a word is unsupported, a transparent fallback—such as fingerspelling or a visible “not available” label—is preferable to inventing a sign. The final display should be reviewed by a competent sign-language consultant before being used for important communication.

6. IMPLEMENTATION DETAILS

6.1 Repository Structure

The project repository separates assets, training data, model weights, core source modules and application entry points. assets/signs contains visual sign resources; assets/audio_cache is described as a location for cached Tamil and English speech clips. data/landmarks_dataset.pickle stores extracted training features. model/gesture_model.p stores the trained classifier. src/ contains the reusable detection, classification, sentence building, translation, TTS and text-to-sign modules.

Top-level scripts provide ways to collect data, train a model, test speech output and launch the applications. requirements.txt lists Python dependencies, and README.md documents the setup and use. The modular layout supports iterative development, but binary pickle files should only be loaded from trusted sources because pickle deserialisation can execute code. For distribution, the project should document how model files were produced and include reproducible training instructions.

6.2 Core Modules

hand_detector.py: captures or receives frames, detects hands and outputs normalised landmark features. It should report when no hand is detected and avoid returning stale coordinates.

gesture_classifier.py: loads the trained model and applies classification logic. It should use the same feature order and normalisation as training. The geometric rules should be documented and covered by unit tests.

sentence_builder.py: keeps track of candidate labels, accepted tokens and sentence editing. It should prevent a single held gesture from being added many times unless repetition is intentionally requested.

translator.py: performs bilingual text mapping and pronunciation/transliteration support. The mapping should be testable independently from the camera.

tts_engine.py: centralises speech generation and playback, including language choice, asynchronous behaviour and error handling.

text_to_sign.py: processes typed or recognised text, creates tokens, looks up visual sign resources and prepares sequential playback. It should expose unsupported tokens rather than silently dropping them.

6.3 Application Entry Points

gui_app.py launches the unified desktop hub, which the README recommends for switching between the two main workflows. streamlit_app.py launches a browser dashboard with project information, webcam snapshot detection, text-to-sign conversion, a bilingual dictionary and a speech-synthesis studio. app.py launches the OpenCV heads-up display with keyboard shortcuts for speech, language selection, spacing, deletion, clearing, automatic speech and quitting. text_to_sign_app.py launches the standalone sign player.

train_model.py is used to train the classifier and cross-validation workflow according to the repository structure. collect_data.py supports collecting custom gesture data. test_tts.py verifies Tamil and English speech output. These scripts should be run from the repository root inside the project virtual environment so relative file paths resolve correctly. Exact command behaviour may change with code revisions; use the README for the version checked out.

6.4 Installation and Execution

The documented setup is: (1) install Python 3.11; (2) clone the repository; (3) change into the repository directory; (4) create and activate a virtual environment; (5) install dependencies; and (6) run the TTS test. On Windows PowerShell, the README gives `py -3.11 -m venv .venv` followed by `\.venv\Scripts\Activate.ps1`.

On Linux or macOS, it gives `python3.11 -m venv .venv` followed by `source .venv/bin/activate`. Then run `pip install -r requirements.txt`.

The unified desktop application can be launched with `python gui_app.py`. The dashboard can be launched with `streamlit run streamlit_app.py`. The webcam HUD can be launched with `python app.py`, and the dedicated player with `python text_to_sign_app.py`. Before a demonstration, verify that the camera and microphone permissions are granted, required speech voices are installed, network-dependent services are reachable and the sign assets/model files exist. Keep a fallback demonstration path in case live camera conditions are poor.

7. TESTING AND EVALUATION

7.1 Test Strategy

Testing should proceed from individual modules to complete user workflows. Unit tests check normalisation, token mapping, sentence editing and sign-resource lookup. Integration tests check that the webcam pipeline produces a valid class label and that the label reaches the sentence builder. End-to-end tests verify that a supported gesture becomes the intended audible phrase, and that typed or spoken text becomes the correct visual sequence.

Figure 7.1. End-to-end testing workflow

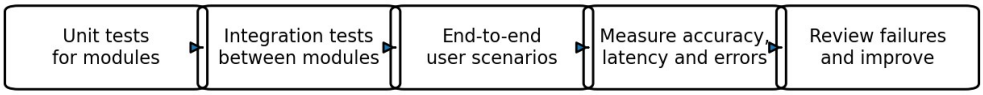


Figure 7.1. End-to-end testing workflow.

Testing must include both positive and negative cases. Positive cases use supported gestures and known dictionary terms. Negative cases include no hand, partially obscured fingers, unknown handshapes, unsupported words, speech-recognition failure, unavailable network access and missing audio devices. User testing should include different users and environmental conditions. Any accuracy figure should identify the dataset size, class distribution, train/test split, participant separation and evaluation method.

7.2 Test Cases

The following test cases are recommended for a formal report. T01: launch the desktop hub and verify both tabs are accessible. T02: detect a clearly visible hand and confirm 21 landmarks are returned. T03: move the same hand around the frame and check that normalised features remain sufficiently stable. T04: hold a known gesture and verify it is accepted once after the confirmation interval. T05: change gesture quickly and verify unstable predictions are not appended. T06: add a space, delete a token and clear the sentence. T07: speak an English sentence and verify English output. T08: repeat in Tamil and verify script/transliteration and speech behaviour. T09: type a supported word and verify the expected sign card. T10: enter an unsupported word and verify that a clear fallback is shown. T11: test play, pause, next, previous and speed adjustment. T12: disable network access and record which features still work.

For each test, record the test environment, input, expected result, actual result, pass/fail status and notes. Do not mark a test as passed based only on the absence of a crash; compare the output against an expected value. A useful report includes screenshots or short recordings for representative cases and a table of failure categories.

7.3 Evaluation Metrics

Classification accuracy is the proportion of correctly classified examples among all evaluated examples. Precision measures the fraction of predicted examples for a class that are correct, while recall measures the fraction of actual examples of that class that are detected. The F1-score combines precision and recall. A confusion matrix shows which gestures are mistaken for one another. Because classes may be imbalanced, macro-averaged precision, recall and F1 can be more informative than accuracy alone.

For the live system, also measure end-to-end latency, false gesture acceptance rate, missed gesture rate and the time required to complete a short message. For speech recognition, word error rate may be calculated from substitutions, deletions and insertions relative to a reference transcript. For text-to-sign, measure token coverage and the correctness of each mapping as reviewed by fluent signers. Usability can be assessed with task completion, error recovery and participant feedback.

7.4 Results Interpretation

The repository’s project report states a 100% test accuracy for its hybrid classifier. This report does not independently verify that number. A high score can be genuine for a small, controlled dataset but may not generalise to new people, camera positions or lighting. If multiple frames from the same recording appear in both training and test sets, the score may overestimate performance because the samples are highly similar. Evaluation should therefore separate participants or recording sessions where possible and publish the class counts and split method.

The system should be evaluated as a full pipeline, not only as a classifier. Even correct gesture labels can produce an incorrect message if sentence boundaries, translation mappings or speech synthesis fail. Likewise, correct speech recognition does not guarantee a correct sign sequence. Results should report each stage independently and include an end-to-end task measure. No experimental results should be invented; replace the proposed test plan with measured values after executing the tests.

8. APPLICATIONS, BENEFITS AND IMPACT

8.1 Potential Applications

The prototype may support simple everyday exchanges in educational settings, homes, demonstrations and public-facing services. A student could communicate a limited set of classroom needs; a family member could use text-to-sign cards to learn a few common expressions; and a project demonstrator could explain the value of computer vision and regional-language support. It may also serve as a foundation for further work in assistive technology, accessible interface design and sign-language resources.

The project should not be deployed as the sole communication method in emergency, medical, legal or financial situations. The vocabulary is limited, and incorrect recognition or mapping can change meaning. In those settings, a qualified interpreter or another verified communication method should remain available.

8.2 Benefits

The use of standard hardware reduces the need for specialised equipment. A modular Python implementation can be studied and extended by students. Bi-directional workflows make the demonstration more complete than a one-way gesture classifier. Tamil and English support acknowledges that accessibility includes regional language needs. The sign player's stepwise display and playback controls can help users review the sequence at their own pace.

The strongest potential benefit is not a claim of universal translation, but a practical platform for exploring communication access. The project makes several technical ideas visible: landmark-based vision, feature normalisation, supervised classification, temporal filtering, language processing, speech synthesis and interactive visualisation.

8.3 Stakeholders and Evaluation

Stakeholders include Deaf and hard-of-hearing users, hearing communication partners, sign-language interpreters, educators, families, developers and accessibility researchers. Their priorities may differ. A developer may focus on accuracy and latency; a Deaf user may prioritise naturalness, grammar, readability and control over how their message is represented. User evaluation should therefore involve people who use the target sign language, not only hearing classmates or developers.

A responsible evaluation plan should recruit participants voluntarily, explain the purpose of the study and avoid collecting unnecessary personal audio or video. Feedback should be used to revise vocabulary, visual presentation, error recovery and language handling. Accessibility success should be judged by whether users can complete a task accurately and comfortably, rather than by the number of features listed in the interface.

9. LIMITATIONS, RISKS AND ETHICS

9.1 Technical Limitations

A limited vocabulary cannot cover unrestricted conversation. Landmark-only features may not represent all dynamic signs, facial expressions, body posture or spatial grammar. Lighting, background clutter, occlusion, camera quality and hand orientation can affect detection. A classifier trained on a small number of people may perform poorly on unseen users. The sentence builder can reduce flicker but may add delay or still accept an incorrect stable prediction.

Text-to-sign mapping is also constrained. Word-by-word conversion can produce a sequence that is understandable as a prompt but unnatural or grammatically incorrect in a sign language. Fingerspelling is not an equivalent replacement for every lexical sign. Speech-to-text may misrecognise names, dates, numbers, accents or code-switching between Tamil and English. TTS quality and availability vary across platforms and network conditions.

9.2 Safety and Communication Risk

A wrong sign-to-audio output can be misunderstood by a hearing recipient. A wrong text-to-sign mapping can cause the Deaf recipient to receive an incorrect instruction. Risk is particularly high for medical dosage, emergency requests, consent, legal information and financial transactions. The application should display that it is a prototype, provide confirmation before speaking sensitive messages and offer an easy way to correct the transcript or sentence.

A low-confidence or unsupported input should trigger a neutral fallback rather than a fabricated output. The system must not imply that it is certified, clinically validated or a substitute for an interpreter. High-stakes use requires substantially stronger testing, domain-specific vocabulary, sign-language expert review and appropriate organisational safeguards.

9.3 Privacy, Consent and Data Handling

Camera and microphone data may contain identifiable information. The application should explain when capture begins, whether audio is sent to a remote service and whether any data is saved. Data collection for training should be voluntary, purpose-limited and protected. Users should be able to stop capture and delete recordings where feasible. Audio caches and logs should avoid storing personal messages unnecessarily.

The use of third-party speech services requires review of their data handling and network requirements. For a classroom demonstration, synthetic sample sentences can reduce exposure of private conversations. Model files should be obtained from trusted project sources, and pickle files should not be loaded from unknown contributors. A future deployment should document retention, access controls and incident response procedures.

9.4 Ethical and Cultural Considerations

Deaf people are not a single homogeneous group. Users differ in preferred language, literacy, region, age and communication style. Sign languages are complete languages with their own structures; they should not be treated as gestures that merely encode spoken words. The project should use respectful terminology and consult Deaf signers throughout design and evaluation.

The repository lists some vocabulary descriptions that are simplified explanations of hand shapes. Those descriptions should be reviewed against the intended sign language and regional conventions before they are presented as authoritative. In particular, the README mentions an ASL alphabet while the broader project context is Tamil/English. The implementation should clearly distinguish American Sign Language fingerspelling from Indian Sign Language and should not imply that the alphabets or signs are interchangeable.

10. FUTURE ENHANCEMENTS

10.1 Expand and Validate the Dataset

Collect examples from multiple consenting participants, left- and right-handed users, different distances and varied lighting. Record class counts and keep a participant-independent test set. Use data augmentation only where it preserves the sign's meaning. Add an explicit unknown class and confidence threshold so that the model can reject gestures outside the vocabulary. Publish a reproducible training procedure and versioned dataset metadata.

10.2 Improve Language and Sign Representation

Develop a structured bilingual lexicon that separates concept identifiers, English terms, Tamil script, transliteration, sign-language glosses and visual assets. Review mappings with fluent signers. Add context-aware token handling for names, numbers, dates and mixed Tamil-English sentences. Instead of converting every token independently, use a meaning representation that can support sign-language word order and grammatical features. Clearly distinguish ISL resources from ASL resources.

10.3 Add Temporal and Multimodal Recognition

Dynamic signs can be better represented by sequences of landmarks rather than a single static feature vector. Future work could evaluate temporal models that learn motion over time, while preserving the current geometric approach as a baseline. Two-hand gestures, body landmarks and facial expression may be needed for broader coverage. Any increase in model complexity should be justified by measured improvements and practical hardware requirements.

10.4 Improve User Experience and Accessibility

Add confidence indicators, transcript correction, a visible unknown-token state, larger sign cards and optional high-contrast mode. Provide keyboard navigation and clear instructions for camera placement. Allow playback speed and repeat controls. Provide captions for audio prompts and visual feedback for microphone state. Conduct usability tests with Deaf users and revise the interface based on observed task failures rather than developer preference alone.

10.5 Deployment, Offline Use and Maintenance

Pin dependency versions, test on clean Windows/Linux/macOS environments and provide a setup script. Cache only non-sensitive static assets. Separate offline features from network-dependent services and show a clear message when connectivity is required. Add automated tests for core modules and continuous integration where practical. Version the model, vocabulary and sign assets together so that a model update does not silently change class meanings.

11. CONCLUSION

11.1 Summary

The Bi-Directional Sign Language & Audio Assistive Suite is a Python-based prototype intended to support communication between signing users and hearing users through two connected workflows. Its forward path uses webcam hand landmarks, normalisation, a hybrid gesture classifier, sentence stabilisation, bilingual text handling and speech synthesis. Its reverse path accepts typed or spoken Tamil/English, tokenises the content, maps tokens to visual sign resources and presents them through a sequential player.

The project's modular source structure and multiple user interfaces provide a useful foundation for academic demonstration. It combines computer vision, machine learning, speech processing, language handling and user-interface design in a socially relevant application. The emphasis on Tamil and English is a meaningful regional-accessibility goal.

11.2 Final Assessment

The system's practical value depends on measured correctness, not just successful execution. The repository reports strong classifier performance, but independent evaluation is needed with representative participants and a participant-separated test set. The project must also be clear about the difference between sign recognition, fingerspelling, word-to-card mapping and genuine sign-language translation. The use of ASL fingerspelling alongside an Indian/Tamil context requires explicit explanation and, ideally, alignment with Indian Sign Language resources.

With expert sign-language review, stronger dataset design, transparent fallback behaviour, privacy-conscious speech handling and systematic end-to-end evaluation, the prototype could develop into a more dependable assistive communication tool. Until then, it should be presented as an educational prototype for limited vocabulary and not as a replacement for professional interpretation in high-stakes settings.

REFERENCES

1. Hepsiba Mark, “Bi-Directional Sign Language & Audio Assistive Suite (Tamil & English),” GitHub repository README. Available: https://github.com/HepsibaMark/sop-project

2. Hepsiba Mark, “Engineering & Academic Project Report: Bi-Directional Sign Language to Audio (Tamil & English) and Text to Sign Language Assistive Suite,” PROJECT_REPORT.md in the project repository. Available: https://github.com/HepsibaMark/sop-project/blob/main/PROJECT_REPORT.md

3. Google, “MediaPipe,” official developer documentation. Available: https://developers.google.com/mediapipe

4. OpenCV, “OpenCV Documentation.” Available: https://docs.opencv.org/

5. scikit-learn developers, “RandomForestClassifier,” scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

6. Streamlit, “Streamlit Documentation.” Available: https://docs.streamlit.io/

7. pyttsx3 contributors, “pyttsx3: Text to Speech in Python,” project documentation. Available: https://pyttsx3.readthedocs.io/

8. World Federation of the Deaf, “Our Work / Human Rights of Deaf People.” Available: https://wfdeaf.org/

9. National Association of the Deaf, “American Sign Language.” Available: https://www.nad.org/resources/american-sign-language/

Note: The repository is the primary source for the system design and listed features. External documentation should be consulted for current library behaviour and installation requirements. Access dates and any additional sources used by the student should be added before submission.

APPENDIX A: SAMPLE USER WORKFLOW

A.1 Sign-to-Audio

1. 1. Open the webcam HUD or the Sign-to-Audio tab in the desktop hub.
2. 2. Select the output language: Tamil or English.
3. 3. Place the hand fully inside the camera frame and wait for landmark detection.
4. 4. Make a supported gesture and hold it until the application confirms the candidate.
5. 5. Check the recognised token in the sentence buffer.
6. 6. Add a space or edit the sentence if needed.
7. 7. Trigger speech output and confirm that the spoken phrase matches the visible text.
8. 8. Clear the buffer before starting a new message.

A.2 Text/Voice-to-Sign

9. 1. Open the dedicated text-to-sign player or the corresponding desktop tab.
10. 2. Choose Tamil or English.
11. 3. Type a short sentence or activate the microphone and speak clearly.
12. 4. Review the transcript and correct names, numbers or misrecognised words.
13. 5. Start conversion and inspect the sequence of sign cards.
14. 6. Use play, pause, next, previous or speed adjustment to follow the sequence.
15. 7. Check any unsupported-token message and use an alternative communication method if the message is important.

APPENDIX B: SUGGESTED TEST CHECKLIST

Area	Check	Result / Evidence
Environment	Python version and dependency installation recorded	
Camera	Camera opens and closes cleanly	
Landmarks	21 landmarks are produced for a visible hand	
No-hand case	No gesture is appended when the hand is absent	
Classification	Every supported class tested with multiple examples	
Stability	Held gesture is accepted once, not on every frame	
Sentence editing	Space, delete and clear work correctly	
English TTS	Output matches visible English sentence	
Tamil TTS	Output matches visible Tamil sentence; voice availability recorded	
Speech input	English and Tamil transcripts checked against references	
Sign mapping	Supported words map to reviewed sign resources	
Unknown token	Unsupported word produces an explicit fallback	
Playback	Play/pause/next/previous/speed controls work	
Network failure	Online-dependent features fail gracefully	
Privacy	Camera/microphone use is visible and controllable	

Complete the Result / Evidence column after executing each test. Include screenshots, sample inputs, actual outputs, error logs and the version of the code tested. Do not present this checklist as completed testing until the checks have actually been performed.